Lecture Note: OOPs Concepts - Polymorphism & Exception Handling

Nishut Suman 29 Dec, 2025 At 10:30 PM Notes Foundation Recommended Resource

Associated Lectures (1)

Description Discussions

Lecture Notes: OOPs Concepts - Polymorphism & Exception Handling

🧩 Prerequisites

Understanding of:

- Python **classes, objects, inheritance**, and **method overriding**
- Basic **function definitions and control flow** (if, for, while)
- Familiarity with **data types** like strings, integers, and lists

🎯 What You'll Be Able to Do

After completing this session, you will be able to:

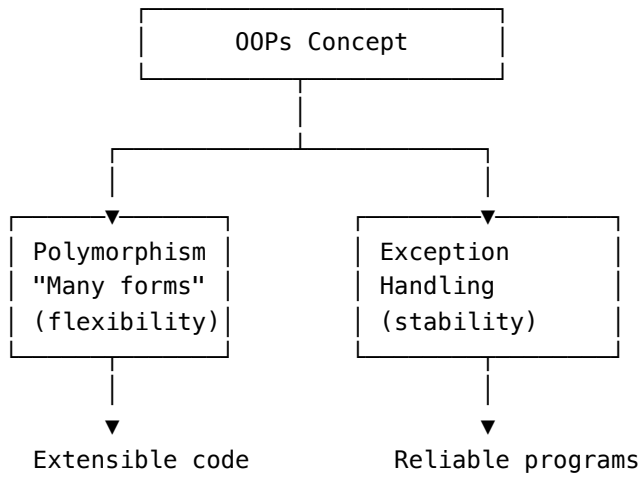
- **Apply** polymorphism to real-world class hierarchies.
- **Use** dunder methods to make your classes behave like Python's built-in types.
- **Design** robust programs with proper exception handling.
- **Combine** flexibility (OOP) and safety (error handling) for professional-grade code.

1. Introduction - Why Mastering This Session Matters

Polymorphism + Exception Handling = **Adaptable + Resilient** programs.

Polymorphism lets many object types share one interface;
exception handling keeps the program remain stable when errors occur.

⚙️ Visual Overview



Real-world example:

When you send an email through Gmail - whether on web, mobile, or desktop - the same **Send** action behaves differently behind the scenes depending on the platform. That's **polymorphism** in action.

If your internet disconnects, Gmail shows a friendly error message instead of crashing - that's **exception handling**, managing unexpected problems gracefully.

---

2. The Foundation: Core Concepts Explained

Concept A - Polymorphism in Shape Hierarchies

```
import math
class Shape:
    def area(self): raise NotImplementedError()
class Circle(Shape):
    def __init__(self,r): self.r=r
    def area(self): return round(math.pi*self.r**2,2)
class Rectangle(Shape):
    def __init__(self,w,h): self.w,self.h=w,h
    def area(self): return self.w*self.h

for s in [Circle(5), Rectangle(4,6)]:
    print(s.__class__.__name__,"Area:",s.area())
```

Output

```
Circle Area: 78.54
Rectangle Area: 24
```

Concept B - Dunder (Magic) Methods

```
class Vector:
    def __init__(self,x,y): self.x,self.y=x,y
    def __add__(self,other): return Vector(self.x+other.x,self.y+other.y)
    def __str__(self): return f"Vector({self.x},{self.y})"
v1,v2=Vector(2,3),Vector(4,1)
print(v1+v2)
```

Output `Vector(6,4)`

Concept C – Exception Handling – Safe Program Flow

```
try:
    with open("data.txt") as f: print(f.read())
except FileNotFoundError:
    print("❌ File not found!")
finally:
    print("✅ File operation complete.")
```

---

3. Worked Examples

Example 1 – Polymorphism with Exception Safety

```
class Device:
    def connect(self): raise NotImplementedError
class Speaker(Device):
    def connect(self): return "🔊 Speaker connected!"
class Headphones(Device):
    def connect(self): return "🎧 Headphones connected!"
class Smartwatch(Device):
    def connect(self): raise ConnectionError("Bluetooth failed.")
for d in [Speaker(),Headphones(),Smartwatch()]:
    try: print(d.connect())
    except ConnectionError as e: print(f"⚠️ {d.__class__.__name__}: {e}")
```

Example 2 – Raising Custom Exceptions

```
class InvalidEmailError(Exception): pass
def create_account(email):
    if "@" not in email:
        raise InvalidEmailError("Invalid email format!")
```

```
    return "✅ Account created"
try:
    print(create_account("vikas.com"))
except InvalidEmailError as e:
    print("Error:", e)
```

---

4. Common Pitfalls & Fixes

- **Catching all errors blindly** → hides bugs → *catch specific ones*
- **Skipping raise NotImplementedError** in base → subclass may silently pass
- **Putting logic in finally** → can overwrite results → reserve it only for cleanup

---

5. Your Turn - Practice & Reflection

Challenge - Smart Shape Manager

Create shapes (**Circle, Square, Triangle**) with **area()**;
handle:

- Invalid/negative inputs
- Unknown shape type Raise custom exceptions for invalid cases.

---

6. Consolidation - Key Takeaways & Next Steps

Core ideas

- Polymorphism = shared interface, many forms
- Dunder methods = extend Python's language features
- Exception handling = stability under error

Next: → *File Handling & Context Managers* Learn how **with** + exceptions ensure safe I/O.

Resources

- Errors and Exceptions
- Python Magic (Dunder) Methods
- Polymorphism in Python

---

7. Additional Topics - Types & Patterns

A. Types of Polymorphism (in Python context)

Type	Description	Example / Keyword
Subtype	Child overrides parent method	Shape.area() → Circle.area()
Ad-hoc	Same operator/function works differently	***add***, singledispatch
Parametric	Works on multiple types via generics	List[T], typing
Duck Typing	Object valid if it “quacks”	any object with .render()

B. Achieving Polymorphism Patterns

Inheritance + Overriding – classic

Duck Typing – capability-based

```
class Button:  def render(self): return "<Button>"
class Text:    def render(self): return "<Text>"
for c in [Button(),Text()]: print(c.render())
```

Abstract Base Classes / Protocols - enforce interface

```
from abc import ABC,abstractmethod
class Payment(ABC):
    @abstractmethod
    def pay(self,amount): ...
```

Function Overloading via **@singledispatch**

```
from functools import singledispatch
@singledispatch
def show(x): return "Unknown"
@show.register(int)  def _(x): return "Int"
@show.register(str)  def _(x): return "Str"
```

C. Types of Exception Handling Behaviors

Specific exceptions → except ValueError:

Multiple exceptions → except (TypeError, ZeroDivisionError):

Generic catch-all → except Exception: *(use rarely)*

Re-raise → raise after partial handling

Chaining → raise NewErr from old

Context managers → with for safe cleanup

Custom hierarchy → class AppError(Exception): ...

D. Managing Exception Types - Best Practices

- Catch **specific** errors.
- Order handlers from most to least specific.
- Keep **finally** for cleanup only.
- Prefer **with** for files/resources.
- Re-raise or chain when needed to preserve context.
- Always log before silencing an error.

E. Integrated Example – Worker Pool with Safe Execution

```
class Job:
    def run(self): raise NotImplementedError
class EmailJob(Job):
    def run(self): raise ConnectionError("SMTP fail")
class BackupJob(Job):
    def run(self): return "Backup done"
for j in [EmailJob(),BackupJob()]:
    try: print(j.run())
    except Exception as e:
        print(f"{j.__class__.__name__} failed: {e}")
```

F. Quick Visual Summary

```
Flexibility → Polymorphism  → Common Interface
Stability   → Exceptions    → Safe Execution
Together    → Robust Systems → Production-ready Code
```

Additional References

- Python Docs – Errors and Exceptions
- functools.singledispatch
- PEP 544 (Protocols / Structural Subtyping)
- Python – Magic Methods
- Wikipedia Polymorphism Diagram
-